

Maddie Masiello
Ryken Thompson

Lab Group A7
EE 329-03 S'25
2025-Apr-23

Introduction:

The goal of this lab was to learn how to use the alternate functions of GPIO pins to enable UART in order to communicate with another device such as a terminal from a laptop. To do this a rudimentary game was created where a player character can move around and collect treasures. The code is fully operational and allows the user to go through both the initial stage in which characters are echoed back to the terminal, and the game stage. In the game stage the borders all are functional allow the character to wrap around to the other side of the screen when crossing. The extra credit functionality of have a multicharacter player character, treasure that turns the character red for 3 turns and solid borders were also implemented. The player character was turned from a single character into characters resembling a stick figure. The treasure functions by activating when the center of the player character is over the treasure, then for the next three moves the character will be red. There are no known bugs in the design. An important lesson learned was how to use the UART interface and properly formatting in the terminal with escape codes as it is very common in embedded systems for debugging and communicating with computers.

Link to YouTube Video Presentation: <https://youtu.be/hw2MHvnhOXY>

(Date of posted video: 05/07/2025)

(Length: 1:59)

Calculations:

$$\text{Baud rate divisors} = \frac{(256) \cdot (f_{clk})}{115 \cdot 10^3} = \frac{(256) \cdot (4 \cdot 10^6)}{115 \cdot 10^3} = 8889$$

Appendix:

Formatted Source Code:

```
-----main.h-----
/*****
 * EE 329 A7
 *****/
 * @file      : main.h
 * @brief     : Main program body and initialization
 * project    : EE 329 S'25 Assignment A7
 * authors    : Maddie Masiello
 * version    : 1
 * date       : 230413
 * compiler   : STM32CubeIDE v.1.12.0 Build: 14980_20230301_1550 (UTC)
 * target     : NUCLEO-L4A6ZG
 * clocks     : 4 MHz MSI to AHB2
 * @attention : (c) 2023 STMicroelectronics. All rights reserved.
 *****/
 *
 *****/
 */

#ifndef __MAIN_H
#define __MAIN_H

#ifdef __cplusplus
extern "C"
{
#endif

/* Includes -----*/
#include "stm32l4xx_hal.h"
#include "uart.h"

/* Exported functions prototypes -----*/
-*/
void SystemClock_Config(void);
void Error_Handler(void);

#ifdef __cplusplus
}
#endif
#endif
```

main.c

```

/*****
 * EE 329 A7
 *****/

 * @file      : main.c
 * @brief     : Main program body and initialization
 * project    : EE 329 S'25 Assignment A7
 * authors    : Maddie Masiello, Ryken Thompson
 * version    : 1.1
 * date       : 250505
 * compiler   : STM32CubeIDE v.1.12.0 Build: 14980_20230301_1550 (UTC)
 * target     : NUCLEO-L4A6ZG
 * clocks     : 4 MHz MSI to AHB2
 * @attention : (c) 2023 STMicroelectronics. All rights reserved.
 *****/

 *
 *****/

#include "main.h"
#include "delay.h"
#include "uart.h"
#include "rand.h"
#include "game_config.h"
#include "game_logic.h"
#include "game_movement.h"
#include "ansi_codes.h"
#include <stdio.h>
#include <stdlib.h>

/**
 * @brief Configures system clocks for 4Mhz
 */
void SystemClock_Config(void) {
    // Configure MSI for 4MHz operation
    RCC->CR |= RCC_CR_MSION; // Enable MSI
    while (!(RCC->CR & RCC_CR_MSIRDY))
        ; // Wait for MSI ready

    RCC->CR &= ~RCC_CR_MSIRANGE; // Clear MSIRANGE
    RCC->CR |= RCC_CR_MSIRGSEL; // Select MSIRANGE from CR
}
```

```

RCC->CR |= RCC_CR_MSIRANGE_6; // Set range 6 for 4MHz

// Configure AHB, APB1, APB2 prescalers
RCC->CFGR &= ~RCC_CFGR_HPRE; // AHB prescaler = 1
RCC->CFGR &= ~RCC_CFGR_PPRE1; // APB1 prescaler = 1
RCC->CFGR &= ~RCC_CFGR_PPRE2; // APB2 prescaler = 1

// Select MSI as system clock
RCC->CFGR &= ~RCC_CFGR_SW; // Clear SW bits
RCC->CFGR |= RCC_CFGR_SW_MSI; // Select MSI as system clock
while ((RCC->CFGR & RCC_CFGR_SWS) != RCC_CFGR_SWS_MSI)
    ; // Wait for MSI
}

void Error_Handler(void);

// Global variables
volatile char charRecv = 0;
volatile int charFlag = 0;

int main(void) {

    //USER PBSW initialization

    //Enable clocks GPIOB, GPIOC, and RNG
    RCC->AHB2ENR |= (RCC_AHB2ENR_GPIOCEN);
    GPIOC->MODER &= ~(GPIO_MODER_MODE13);

    //Clear the OTyPER registers for PC0, PC1, PC2, and PC3
    //(0b: Output push-pull)
    GPIOC->OTYPER &= ~(GPIO_OTYPER_OT13);

    //Clear the PUPDR registers for PC13
    //(00b: No pull-up, pull-down)
    GPIOC->PUPDR &= ~(GPIO_PUPDR_PUPD0 | GPIO_PUPDR_PUPD13);

    //Set pull down on PC13
    GPIOC->PUPDR |= (GPIO_PUPDR_PUPD13_1);

    //Initializations
    HAL_Init();
    SystemClock_Config();
    LPUART_init();

```

```

RNG_init();

// Implement escape sequence requirements
LPUART_Print(ANSI_CLEAR_SCREEN); // Clear screen once at start
LPUART_Print("\x1B[3B");        // Move cursor down 3 lines
LPUART_Print("\x1B[5C");        // Move cursor right 5 spaces
LPUART_Print("All good students read the");
LPUART_Print("\x1B[1B");        // Move cursor down 1 line
LPUART_Print("\x1B[21D");       // Move cursor left 21 spaces
LPUART_Print(ANSI_BLINK);       // Enable blinking text
LPUART_Print("Reference Manual");
LPUART_Print("\x1B[H");         // Move cursor to top left
LPUART_Print(ANSI_RESET_ATTRS); // Remove character attributes
LPUART_Print("Input: ");

//Await PBSW
while (!(GPIOC->IDR & (GPIO_PIN_13))) {
    if (charFlag) {
        char str[2];
        charFlag = 0;
        str[0] = charRecv;
        str[1] = '\0';
        LPUART_Print(str);
    }
}

splash_screen(); // Show splash screen

LPUART_Print(ANSI_CLEAR_SCREEN); // Clear screen once at start
LPUART_Print(ANSI_HIDE_CURSOR);  // Hide cursor
draw_boundary();                  // Draw the game boundary
init_character_position();        // Initialize character position
treasure_init();
draw_treasure();
LPUART_Print(ANSI_COLOR_WHITE);
update_position();               // Show initial X position

// Game variables
int treasure_turn_count = 0;
int current_char_color = 0;

// Character color array
const char *char_colors[] = {ANSI_COLOR_WHITE,

```

```

ANSI_COLOR_RED,
ANSI_COLOR_GREEN,
ANSI_COLOR_BLUE});

while (1) {
    if (charFlag) {          // check if a character was received
        charFlag = 0;        // reset the flag
        erase_position();     // erase the current position

        switch (charRecv) {
            // Handle color changes
            case 'R':
                current_char_color = 1; // RED
                break;
            case 'G':
                current_char_color = 2; // GREEN
                break;
            case 'B':
                current_char_color = 3; // BLUE
                break;
            case 'W':
                current_char_color = 0; // WHITE
                break;

            // Handle character movement
            // Decrease turn counter for treasure detection if needed
            case 'w':
                move_character(MOVE_UP);
                if (treasure_turn_count>0) treasure_turn_count--;
                break;
            case 's':
                move_character(MOVE_DOWN);
                if (treasure_turn_count>0) treasure_turn_count--;
                break;
            case 'a':
                move_character(MOVE_LEFT);
                if (treasure_turn_count>0) treasure_turn_count--;
                break;
            case 'd':
                move_character(MOVE_RIGHT);
                if (treasure_turn_count>0) treasure_turn_count--;
                break;

            // Redraw everything if needed
            case 'r':

```

```

        LPUART_Print(ANSI_CLEAR_SCREEN);
        draw_boundary();
        break;

//Default: do nothing
}

//Check for treasure hit
if (detect_treasure()) {
    treasure_turn_count = 3; //Turn red for 3 turns
}

draw_treasure(); // In case player overwrites without deactivating

//Color handling
if (treasure_turn_count > 0 ) {
    LPUART_Print(ANSI_COLOR_RED);
} else {
    LPUART_Print(char_colors[current_char_color]);
}

update_position(); // update the cursor position
}
}
}

/**
 * @brief This function is executed in case of error occurrence.
 * @retval None
 */
void Error_Handler(void) {
    /* USER CODE BEGIN Error_Handler_Debug */
    /* User can add his own implementation to report the HAL error return state */
    __disable_irq();
    while (1) {
    }
}

#ifdef USE_FULL_ASSERT
void assert_failed(uint8_t *file, uint32_t line)
{
}
#endif

```

uart.h

```
/* *****
 * EE 329 A7
 * *****
 * @file      : uart.h
 * @brief     : UART functions and display handling
 * project    : EE 329 S'25 Assignment A7
 * authors    : Maddie Masiello
 * version    : 1
 * date       : 230413
 * compiler   : STM32CubeIDE v.1.12.0 Build: 14980_20230301_1550 (UTC)
 * target     : NUCLEO-L4A6ZG
 * clocks     : 4 MHz MSI to AHB2
 * @attention : (c) 2023 STMicroelectronics. All rights reserved.
 * *****
 *
 * *****
 */
```

```
#ifndef UART_H
```

```
#define UART_H
```

```
#include "main.h"
```

```
/* UART Constants */
```

```
#define UART_BAUD_DIV 8889 // 4 MHz MSI / 115200 baud
```

```
#define GPIO_AF_LPUART 8 // AF8 mode for LPUART
```

```
/* UART Communication Functions */
```

```
void LPUART_init(void);
```

```
void LPUART_Print(const char *message);
```

```
void LPUART_ESC_Print(const char* esc_sequence, const char* text);
```

```
/* Interrupt Variables */
```

```
extern volatile char charRecv;
```

```
extern volatile int charFlag;
```

```
#endif
```

uart.c

```
/* *****
```



```

* EE 329 A7
*****
* @file          : uart.c
* @brief         : UART functions and display handling
* project        : EE 329 S'25 Assignment A7
* authors        : Maddie Masiello
* version        : 1
* date           : 230413
* compiler       : STM32CubeIDE v.1.12.0 Build: 14980_20230301_1550 (UTC)
* target         : NUCLEO-L4A6ZG
* clocks         : 4 MHz MSI to AHB2
* @attention     : (c) 2023 STMicroelectronics. All rights reserved.
*****
*
*****
*/

#include "uart.h"
#include "delay.h"
#include "game_config.h"
#include "ansi_codes.h"
#include "game_movement.h"
#include <stdio.h>

void LPUART_init(void)
{
    PWR->CR2 |= (PWR_CR2_IOSV); // power avail on PG[15:2] (LPUART1)
    RCC->AHB2ENR |= (RCC_AHB2ENR_GPIOGEN); // enable GPIOG clock
    RCC->APB1ENR2 |= RCC_APB1ENR2_LPUART1EN; // enable LPUART clock bridge
    /* USER: configure GPIOG registers MODER/PUPDR/OTYPER/OSPEEDR then
    select AF mode and specify which function with AFR[0] and AFR[1] */

    // Set PG7 and PG8 to Alternate Function mode (AF8)
    GPIOG->MODER &= ~(GPIO_MODER_MODE7 | GPIO_MODER_MODE8);
    GPIOG->MODER |= (GPIO_MODER_MODE7_1 | GPIO_MODER_MODE8_1); // AF mode

    // Select AF8 for PG7 and PG8
    GPIOG->AFR[0] &= ~(0xF << GPIO_AFR_L_AFSEL7_Pos); // clear bits for PG7
    // AF8 = LPUART1_TX
    GPIOG->AFR[0] |= (GPIO_AF_LPUART << GPIO_AFR_L_AFSEL7_Pos);
    GPIOG->AFR[1] &= ~(0xF << (8 - 8) * 4); // clear bits for PG8
    GPIOG->AFR[1] |= (GPIO_AF_LPUART << (8 - 8) * 4); // AF8 = LPUART1_RX

    // Push-pull output, high speed, no pull-up/down
    GPIOG->OTYPER &= ~(GPIO_OTYPER_OT7 | GPIO_OTYPER_OT8);

```

```

    GPIOG->OSPEEDR |= (GPIO_SPEED_FREQ_VERY_HIGH << GPIO_OSPEEDR_OSPEED7_Pos) |
(GPIO_SPEED_FREQ_VERY_HIGH << GPIO_OSPEEDR_OSPEED8_Pos);
    GPIOG->PUPDR &= ~(GPIO_PUPDR_PUPD7 | GPIO_PUPDR_PUPD8); // No PUPD

    LPUART1->CR1 &= ~(USART_CR1_M1 | USART_CR1_M0); // 8-bit data
    LPUART1->BRR = UART_BAUD_DIV; // 4 MHz MSI / 115200 baud
    LPUART1->CR1 |= USART_CR1_UE; // enable LPUART1
    LPUART1->CR1 |= (USART_CR1_TE | USART_CR1_RE); // enable xmit & rcv
    LPUART1->CR1 |= USART_CR1_RXNEIE; // enable LPUART1 rcv interrupt
    LPUART1->ISR &= ~(USART_ISR_RXNE); // clear Recv-Not-Empty flag
    /* USER: set baud rate register (LPUART1->BRR) */
    NVIC->ISER[2] = (1 << (LPUART1_IRQn & 0x1F)); // enable LPUART1 ISR
    __enable_irq(); // enable global interrupts
}

```

```

void LPUART_Print(const char *message)
{
    uint16_t iStrIdx = 0;
    while (message[iStrIdx] != 0)
    {
        while (!(LPUART1->ISR & USART_ISR_TXE)) // wait for empty xmit buffer
            ;
        LPUART1->TDR = message[iStrIdx]; // send this character
        iStrIdx++; // advance index to next char
    }
}

```

```

void LPUART1_IRQHandler(void)
{
    if (LPUART1->ISR & USART_ISR_RXNE)
    {
        charRecv = LPUART1->RDR;
        charFlag = 1;

        // Handle color changes
        switch(charRecv) {
            case 'R':
                LPUART_Print(ANSI_COLOR_RED);
                break;
            case 'G':
                LPUART_Print(ANSI_COLOR_GREEN);
                break;
            case 'B':
                LPUART_Print(ANSI_COLOR_BLUE);
                break;
        }
    }
}

```

```

        case 'W':
            LPUART_Print(ANSI_COLOR_WHITE);
            break;
    }
}

void LPUART_ESC_Print(const char *esc_sequence, const char *text)
{
    LPUART_Print(esc_sequence);
    LPUART_Print(text);
}

void splash_screen(void)
{
    LPUART_Print(ANSI_CLEAR_SCREEN);
    LPUART_Print(ANSI_HIDE_CURSOR);

    const char* colors[] = {
        ANSI_COLOR_RED,
        ANSI_COLOR_GREEN,
        ANSI_COLOR_BLUE,
        ANSI_COLOR_CYAN,
        ANSI_COLOR_YELLOW,
        ANSI_COLOR_MAGENTA
    };

    // ASCII Art title - centered at row 10
    const char* title[] = {
        " _____",
        "| | | | | / _ \\ / _ \\",
        "| | | | | \\ \\ ) | ( ) |",
        "| | | | | / _ \\ / _ \\",
        "| | | | | / _ \\ / _ \\",
        "      / _ \\ | | |",
        "      / _ \\ / _ \\",
        "      / _ \\ / _ \\",
        "      / _ \\ / _ \\",
        "      / _ \\ / _ \\",
    };

    const int title_height = 9;
    const int title_start_row = 10;

    // Display and animate title with color cycling (2.5 seconds)
    for (int cycle = 0; cycle < 5; cycle++) {

```

```

// Position and print title with current color
LPUART_Print(colors[cycle % 6]);
LPUART_Print(ANSI_BOLD);

for (int i = 0; i < title_height; i++) {
    char pos[16];
    sprintf(pos, "\x1B[%d;20H", title_start_row + i);
    LPUART_Print(pos);
    LPUART_Print(title[i]);
}

LPUART_Print(ANSI_RESET_ATTRS);
delay_us(500000); // 0.5s per color
}

// Fireworks animation (2.5 seconds)
const char* firework_chars = "*+.x";
const int firework_count = 5;
// {row,col}
const int positions[5][2] = {{8,10}, {8,70}, {20,40}, {30,20}, {30,60}};

for (int frame = 0; frame < 5; frame++) {
    for (int f = 0; f < firework_count; f++) {
        if ((frame + f) % 2 == 0) {
            char pos[16];
            sprintf(pos, "\x1B[%d;%dH", positions[f][0], positions[f][1]);
            LPUART_Print(pos);
            LPUART_Print(colors[f % 6]);

            // Draw firework burst pattern
            for (int i = 0; i < 8; i++) {
                char burst_pos[16];
                int offset_row = (i < 4) ? -1 : (i < 6 ? 0 : 1);
                int offset_col = ((i % 4) < 2) ? -1 : 1;
                sprintf(burst_pos, "\x1B[%d;%dH%c",
                    positions[f][0] + offset_row,
                    positions[f][1] + offset_col,
                    firework_chars[i % 4]);
                LPUART_Print(burst_pos);
            }
        }
    }
    LPUART_Print(ANSI_RESET_ATTRS);
    delay_us(500000); // 0.5s between firework frames
}

```

```

    delay_us(500000); // Final pause before game starts
    LPUART_Print(ANSI_CLEAR_SCREEN);
}

void update_position()
{
    int row, col;
    get_character_position(&row, &col);

    LPUART_Print(ANSI_BOLD); // make the char bolder
    char str[ESC_BUFFER_SIZE];
    sprintf(str, "\x1B[%d;%dH", row, col); // able to see the char move
    LPUART_Print(str);
    LPUART_Print("X");
    LPUART_Print("\x1B[22m"); // Turn off bold while preserving color
}

void erase_position(void)
{
    int row, col;
    get_character_position(&row, &col);

    char esc[ESC_BUFFER_SIZE];
    sprintf(esc, "\x1B[%d;%dH ", row, col); // Position cursor and erase with
space in one sequence
    LPUART_Print(esc);
}

void draw_boundary(void)
{
    char pos[POS_BUFFER_SIZE];

    // Reset text attributes
    LPUART_Print(ANSI_RESET_ATTRS);

    // Draw top and bottom borders (row 1 and 40)
    for (int c = BOUNDARY_LEFT; c <= BOUNDARY_RIGHT; c++)
    {
        // Top border
        sprintf(pos, "\x1B[1;%dH%s", c, BORDER_HORIZONTAL, ANSI_RESET_ATTRS);
        LPUART_Print(pos);

        // Bottom border
        sprintf(pos, "\x1B[40;%dH%s", c, BORDER_HORIZONTAL, ANSI_RESET_ATTRS);
    }
}

```

```

        LPUART_Print(pos);
    }

    // Draw left and right borders (column 1 and 80)
    for (int r = PLAY_AREA_TOP; r <= PLAY_AREA_BOTTOM; r++)
    {
        // Left border
        sprintf(pos, "\x1B[%d;1H%s", r, BORDER_VERTICAL, ANSI_RESET_ATTRS);
        LPUART_Print(pos);

        // Right border
        sprintf(pos, "\x1B[%d;80H%s", r, BORDER_VERTICAL, ANSI_RESET_ATTRS);
        LPUART_Print(pos);
    }

    // Draw corners with reset after each
    char corner[ESC_BUFFER_SIZE];

    sprintf(corner, "\x1B[1;1H%s", BORDER_TOP_LEFT, ANSI_RESET_ATTRS);
    LPUART_Print(corner);

    sprintf(corner, "\x1B[1;80H%s", BORDER_TOP_RIGHT, ANSI_RESET_ATTRS);
    LPUART_Print(corner);

    sprintf(corner, "\x1B[40;1H%s", BORDER_BOTTOM_LEFT, ANSI_RESET_ATTRS);
    LPUART_Print(corner);

    sprintf(corner, "\x1B[40;80H%s", BORDER_BOTTOM_RIGHT, ANSI_RESET_ATTRS);
    LPUART_Print(corner);
}

```

-----game_movement.h-----

```

/*****
 * EE 329 A7
 *****/

 * @file           : game_movement.h
 * @brief          : Game cursor movement and control functions
 * project         : EE 329 S'25 Assignment A7
 * authors         : Maddie Masiello, Ryken Thompson
 * version        : 1
 * date           : 230413
 * compiler        : STM32CubeIDE v.1.12.0 Build: 14980_20230301_1550 (UTC)
 * target         : NUCLEO-L4A6ZG
 * clocks         : 4 MHz MSI to AHB2

```

```

* @attention      : (c) 2023 STMicroelectronics. All rights reserved.
*****
*
*****
*/

#ifndef GAME_MOVEMENT_H
#define GAME_MOVEMENT_H

#include <stdint.h>
#include "game_config.h"

/* Movement Types */
typedef enum
{
    MOVE_UP,
    MOVE_DOWN,
    MOVE_LEFT,
    MOVE_RIGHT
} MovementDirection;

/* Border Types */
typedef enum
{
    BORDER_TYPE_HORIZONTAL,
    BORDER_TYPE_VERTICAL
} BorderType;

/* Function Declarations */
/**
 * @brief Updates character position based on direction
 * @param direction Movement direction (MOVE_UP, MOVE_DOWN, etc.)
 */
void move_character(MovementDirection direction);

/**
 * @brief Gets current character position
 * @param row Pointer to store row position
 * @param col Pointer to store column position
 */
void get_character_position(int *row, int *col);

void init_character_position(void);

/** Movement state tracking */

```

```
//extern int current_row;
//extern int current_col;

#endif /* GAME_MOVEMENT_H */
```

-----game_movement.c-----

```

/*****
 * EE 329 A7
 *****/

 * @file      : game_movement.c
 * @brief     : Game cursor movement and control functions
 * project    : EE 329 S'25 Assignment A7
 * authors    : Maddie Masiello, Ryken Thompson
 * version    : 1
 * date       : 250505
 * compiler   : STM32CubeIDE v.1.12.0 Build: 14980_20230301_1550 (UTC)
 * target     : NUCLEO-L4A6ZG
 * clocks     : 4 MHz MSI to AHB2
 * @attention : (c) 2023 STMicroelectronics. All rights reserved.
 *****/

 *
 *****/

#include "game_movement.h"
#include "uart.h"
#include "ansi_codes.h"
#include <stdio.h>

/* Movement state */
int current_row;
int current_col;

/**
 * @brief Initialize character position
 */
void init_character_position(void) {
    current_row = INITIAL_ROW;
    current_col = INITIAL_COL;
}

```



```

/**
 * @brief Change row and col to match character movement direction
 * @param direction Direction to move player character
 */
void move_character(MovementDirection direction) {
    switch (direction) {
        case MOVE_UP:
            current_row--;
            //Check for boundaries and wrap around if hit
            if (current_row < PLAY_AREA_TOP+1) {
                current_row = PLAY_AREA_BOTTOM-1; // Wrap around
            }
            break;

        case MOVE_DOWN:
            current_row++;
            //Check for boundaries and wrap around if hit
            if (current_row > PLAY_AREA_BOTTOM-1) {
                current_row = PLAY_AREA_TOP+1; // Wrap around
            }
            break;

        case MOVE_LEFT:
            current_col--;
            //Check for boundaries and wrap around if hit
            if (current_col < PLAY_AREA_LEFT+1) {
                current_col = PLAY_AREA_RIGHT-1; // Wrap around
            }
            break;

        case MOVE_RIGHT:
            current_col++;
            //Check for boundaries and wrap around if hit
            if (current_col > PLAY_AREA_RIGHT-1) {
                current_col = PLAY_AREA_LEFT+1; // Wrap around
            }
            break;
    }
}

/**
 * @brief Gets current character position
 * @param row Pointer to store row position
 * @param col Pointer to store column position
 */

```

```

void get_character_position(int *row, int *col) {
    *row = current_row;
    *col = current_col;
}

```

-----game_logic.h-----

```

/*****
 * EE 329 A7
 *****/

* @file      : game_logic.h
* @brief     : Game cursor movement and control functions
* project    : EE 329 S'25 Assignment A7
* authors    : Maddie Masiello, Ryken Thompson
* version    : 1
* date       : 230413
* compiler   : STM32CubeIDE v.1.12.0 Build: 14980_20230301_1550 (UTC)
* target     : NUCLEO-L4A6ZG
* clocks     : 4 MHz MSI to AHB2
* @attention : (c) 2023 STMicroelectronics. All rights reserved.
*****/

*

*****/

*/

#ifndef INC_GAME_LOGIC_H_
#define INC_GAME_LOGIC_H_

#include "main.h"

/* Display Functions */
void draw_boundary(void);
void splash_screen(void);
void update_position(void);
void erase_position(void);

/* Treasure Functions */
void treasure_init(void);
int detect_treasure(void);
void draw_treasure(void);

#endif /* INC_GAME_LOGIC_H_ */

```

game_logic.c

```
/*
*****
* EE 329 A7
*****
* @file      : game_logic.c
* @brief     : Game logic and treasure placement
* project    : EE 329 S'25 Assignment A7
* authors    : Maddie Masiello, Ryken Thompson
* version    : 1.0
* date       : 250505
* compiler   : STM32CubeIDE v.1.12.0 Build: 14980_20230301_1550 (UTC)
* target     : NUCLEO-L4A6ZG
* clocks     : 4 MHz MSI to AHB2
* @attention : (c) 2025 STMicroelectronics. All rights reserved.
*****
*
*****
*/
```

```
#include "game_logic.h"
#include "game_config.h"
#include "ansi_codes.h"
#include "game_movement.h"
#include "rand.h"
#include "uart.h"
#include "delay.h"
#include <stdio.h>
#include <stdlib.h>
```

```
/**
 * @brief Prints the game initialization splash screen to LPUART1
 */
```

```
void splash_screen(void) {
    //Prepare connected terminal for splash screen
    LPUART_Print(ANSI_CLEAR_SCREEN);
    LPUART_Print(ANSI_HIDE_CURSOR);

    // Color cycle array
    const char *colors[] = {
        ANSI_COLOR_RED,
        ANSI_COLOR_GREEN,
        ANSI_COLOR_BLUE,
```

```
ANSI_COLOR_CYAN,  
ANSI_COLOR_YELLOW,  
ANSI_COLOR_MAGENTA };
```

```
const char *title[] = {
    " _____ ",
    "| | | / \ \ / _ \ \ ",
    "| _| | _ \ \ _ ) | ( _ | ",
    "| | | | _ _ ) / _/ \ \ , | ",
    "| | | | _ / _ | _ / _ ",
    "      / \ \ | _ | ",
    "      / _ \ \ / / ",
    "      / _ \ \ / / ",
    "      / _ \ \ \ \ / _ " };
```

```
const int title_height = 9;  
const int title_start_row = 10;
```

```
// Display and animate title with color cycling (2.5 seconds)
```

```
for (int cycle = 0; cycle < 5; cycle++) {
    // Position and print title with current color
    LPUART_Print(colors[cycle % 6]);
    LPUART_Print(ANSI_BOLD);

    for (int i = 0; i < title_height; i++) {
        char pos[16];
        sprintf(pos, "\x1B[%d;20H", title_start_row + i);
        LPUART_Print(pos);
        LPUART_Print(title[i]);
    }
}
```

```
LPUART_Print(ANSI_RESET_ATTRS);
delay_us(500000); // 0.5s per color
}
```

```
// Fireworks animation (2.5 seconds)
```

```
const char *firework_chars = "+.x";
const int firework_count = 5;
const int positions[5][2] = { { 8, 10 }, { 8, 70 }, { 20, 40 }, { 30, 20 }, {
    30, 60 } }; // {row,col}
```

```
//Five firework positions around the splash screen
```

```
for (int frame = 0; frame < 5; frame++) {  
    for (int f = 0; f < firework_count; f++) {
```

```

        if ((frame + f) % 2 == 0) {
            char pos[16];
            sprintf(pos, "\x1B[%d;%dH", positions[f][0], positions[f][1]);
            LPUART_Print(pos);
            LPUART_Print(colors[f % 6]);

            // Draw firework burst pattern
            for (int i = 0; i < 8; i++) {
                char burst_pos[16];
                int offset_row = (i < 4) ? -1 : (i < 6 ? 0 : 1);
                int offset_col = ((i % 4) < 2) ? -1 : 1;
                sprintf(burst_pos, "\x1B[%d;%dH%c", positions[f][0] + offset_row,
                    positions[f][1] + offset_col, firework_chars[i % 4]);
                LPUART_Print(burst_pos);
            }
        }
        LPUART_Print(ANSI_RESET_ATTRS);
        delay_us(500000); // 0.5s between firework frames
    }

    delay_us(500000); // Final pause before game starts
    LPUART_Print(ANSI_CLEAR_SCREEN);
}

/**
 * @brief Updates the characters position on LPUART terminal
 */
void update_position() {
    int row, col;
    get_character_position(&row, &col);

    LPUART_Print(ANSI_BOLD); // make the char bolder
    char str[ESC_BUFFER_SIZE];

    //Move cursor to head position
    sprintf(str, "\x1B[%d;%dH", (uint8_t) (row - 1), (uint8_t) (col - 1));
    LPUART_Print(str);

    LPUART_Print(" O "); // Print head

    //Move cursor to torso position
    sprintf(str, "\x1B[%d;%dH", (uint8_t) (row), (uint8_t) (col - 1));
    LPUART_Print(str);

```

```

    LPUART_Print("/|\\"); // Print torso

    //Move cursor to legs position
    sprintf(str, "\x1B[%d;%dH", (uint8_t) (row + 1), (uint8_t) (col - 1));
    LPUART_Print(str);

    LPUART_Print("/ \\"); // Print legs

    LPUART_Print("\x1B[22m"); // Turn off bold while preserving color
}

/**
 * @brief Erases character on UART terminal by replacing with a space
 */
void erase_position(void) {
    int row, col;
    get_character_position(&row, &col);

    char esc[ESC_BUFFER_SIZE];

    // Position cursor and erase head with spaces in one sequence
    sprintf(esc, "\x1B[%d;%dH  ", (uint8_t) (row - 1), (uint8_t) (col - 1));
    LPUART_Print(esc);

    // Position cursor and erase torso with spaces in one sequence
    sprintf(esc, "\x1B[%d;%dH  ", (uint8_t) (row), (uint8_t) (col - 1));
    LPUART_Print(esc);

    // Position cursor and erase legs with spaces in one sequence
    sprintf(esc, "\x1B[%d;%dH  ", (uint8_t) (row + 1), (uint8_t) (col - 1));
    LPUART_Print(esc);
}

/**
 * @brief Draw boundary for game on UART terminal 80x40 char
 */
void draw_boundary(void) {
    char pos[POS_BUFFER_SIZE];

    // Reset text attributes
    LPUART_Print(ANSI_RESET_ATTRS);

    // Draw top and bottom borders (row 1 and 40)
    for (int c = BOUNDARY_LEFT; c <= BOUNDARY_RIGHT; c++) {
        // Top border

```

```

    sprintf(pos, "\x1B[1;%dH%s", c, BORDER_HORIZONTAL, ANSI_RESET_ATTRS);
    LPUART_Print(pos);

    // Bottom border
    sprintf(pos, "\x1B[40;%dH%s", c, BORDER_HORIZONTAL, ANSI_RESET_ATTRS);
    LPUART_Print(pos);
}

// Draw left and right borders (column 1 and 80)
for (int r = PLAY_AREA_TOP; r <= PLAY_AREA_BOTTOM; r++) {
    // Left border
    sprintf(pos, "\x1B[%d;1H%s", r, BORDER_VERTICAL, ANSI_RESET_ATTRS);
    LPUART_Print(pos);

    // Right border
    sprintf(pos, "\x1B[%d;80H%s", r, BORDER_VERTICAL, ANSI_RESET_ATTRS);
    LPUART_Print(pos);
}

// Draw corners with reset after each
char corner[ESC_BUFFER_SIZE];

    sprintf(corner, "\x1B[1;1H%s", BORDER_TOP_LEFT, ANSI_RESET_ATTRS); // Top-
left
    LPUART_Print(corner);

    sprintf(corner, "\x1B[1;80H%s", BORDER_TOP_RIGHT, ANSI_RESET_ATTRS); // Top-
right
    LPUART_Print(corner);

    sprintf(corner, "\x1B[40;1H%s", BORDER_BOTTOM_LEFT, ANSI_RESET_ATTRS); //
Bottom-left
    LPUART_Print(corner);

    sprintf(corner, "\x1B[40;80H%s", BORDER_BOTTOM_RIGHT, ANSI_RESET_ATTRS); //
Bottom-right
    LPUART_Print(corner);
}

// Treasure object
typedef struct {
    int row;
    int col;
    int active;
} treasure_t;

```

```

treasure_t treasure[TREASURE_NUM];

/**
 * @brief Check treasures for duplicate positions during initialization
 */
int is_duplicate(treasure_t positions[], int count, int row, int col) {
    for (int i = 0; i < count; i++) {
        if (positions[i].row == row && positions[i].col == col) {
            return 1; // duplicate found
        }
    }
    return 0;
}

/**
 * @brief Initialize treasure positions for game
 */
void treasure_init() {
    // Create 6 treasures at 6 different random positions within border
    int count = 0;

    while (count < TREASURE_NUM) {
        //Find random treasure positions within boundaries

        int row = RNG_genr(MAX_ROWS-1, 1);
        int col = RNG_genr(MAX_COLS-1, 1);

        // Verify treasure position has not already been used
        if (!is_duplicate(treasure, count, row, col)) {
            treasure[count].row = row;
            treasure[count].col = col;
            treasure[count].active = 1;
            count++;
        }
    }

    // Draw treasure on UART terminal
    draw_treasure();
}

/**
 * @brief Detection of treasure and player collisions
 * @ret 1 - true, 0 - false
 */

```



```

int detect_treasure() {
    int row, col;
    get_character_position(&row, &col);

    // Check all treasure places for collision
    for (int i = 0; i < TREASURE_NUM; i++) {
        //Make sure current treasure is active
        if (!treasure[i].active) continue;

        //Check for collision with center of player character
        if ((row == treasure[i].row) && (col == treasure[i].col)) {
            //Match
            //Disable treasure
            treasure[i].active = 0;
            return 1;
        }
    }
    // No treasure detected
    return 0;
}

/**
 * @brief Drawing of active treasures on UART terminal
 */
void draw_treasure(void) {

    //change color to gold
    LPUART_Print(ANSI_COLOR_YELLOW);

    for (int i = 0; i < TREASURE_NUM; i++) {
        //Don't draw inactive treasures
        if (!treasure[i].active) continue;

        int row = treasure[i].row;
        int col = treasure[i].col;

        //Move cursor to treasure position
        char str[ESC_BUFFER_SIZE];
        sprintf(str, "\x1B[%d;%dH", row, col);
        LPUART_Print(str);

        LPUART_Print("X"); // Print treasure - X marks the spot
    }
}

```

-----ansi_codes.h-----

```
/* *****  
 * EE 329 A7  
 * *****  
 * @file      : ansi_codes.h  
 * @brief     : ANSI escape sequence definitions  
 * project    : EE 329 S'25 Assignment A7  
 * authors    : Maddie Masiello  
 * version    : 1  
 * date       : 230413  
 * compiler   : STM32CubeIDE v.1.12.0 Build: 14980_20230301_1550 (UTC)  
 * target     : NUCLEO-L4A6ZG  
 * clocks     : 4 MHz MSI to AHB2  
 * @attention : (c) 2023 STMicroelectronics. All rights reserved.  
 * *****  
 *  
 * *****  
 */
```

```
#ifndef ANSI_CODES_H  
#define ANSI_CODES_H
```

```
/* Screen Control */  
#define ANSI_CLEAR_SCREEN "\x1B[2J"  
#define ANSI_HIDE_CURSOR "\x1B[?25l"  
#define ANSI_RESET_ATTRS "\x1B[0m"  
#define ANSI_BOLD "\x1B[1m"
```

```
/* Colors */  
#define ANSI_COLOR_RED "\x1B[31m"  
#define ANSI_COLOR_GREEN "\x1B[32m"  
#define ANSI_COLOR_BLUE "\x1B[34m"  
#define ANSI_COLOR_WHITE "\x1B[37m"  
#define ANSI_COLOR_CYAN "\x1B[36m"  
#define ANSI_COLOR_YELLOW "\x1B[33m"  
#define ANSI_COLOR_MAGENTA "\x1B[35m"  
#define ANSI_BLINK "\x1B[5m"
```

```
/* Border Characters */  
#define BORDER_HORIZONTAL "—"   
#define BORDER_VERTICAL " | "  
#define BORDER_TOP_LEFT "┌"
```

```
#define BORDER_TOP_RIGHT "┐"
#define BORDER_BOTTOM_LEFT "└"
#define BORDER_BOTTOM_RIGHT "┘"

/* Cursor Movement */
#define ANSI_CURSOR_POS(row, col) "\x1B[" #row ";" #col "H"

#endif /* ANSI_CODES_H */
```